

JUSTIFICACIÓN DEL USO DE PATRONES DE DISEÑO

En el desarrollo del sistema de gestión de eventos y venta de entradas se seleccionaron distintos patrones de diseño con el objetivo de resolver problemas específicos de arquitectura, mejorar la escalabilidad del sistema y cumplir con los requerimientos funcionales definidos en el proyecto. A continuación, se justifica su uso indicando el requisito funcional (RF) asociado, el problema que resuelve y la razón de su elección frente a alternativas más simples.

Singleton

RF-045, RF-048 (gestión global del sistema y control de acceso a datos)

Se utilizó Singleton para garantizar una única instancia global encargada de la gestión del sistema.

Se eligió en lugar de múltiples instancias de control o variables globales, ya que estas pueden generar inconsistencias en el estado del sistema, duplicación de información y problemas de sincronización. Tampoco se consideró necesario el uso de frameworks de inyección de dependencias debido al alcance académico del proyecto.

Factory Method

RF-013, RF-023 (creación y gestión de eventos)

Se utilizó Factory Method para la creación de eventos de manera estructurada.

Se eligió en lugar de la instanciación directa con `new` o condicionales (`if-else`) para seleccionar tipos de evento, ya que estas alternativas generan acoplamiento fuerte y dificultan la extensión del sistema cuando se agregan nuevos tipos de eventos.

Prototype

RF-046 (generación de reportes reutilizables)

Se utilizó Prototype para la creación de objetos mediante clonación.

Se eligió en lugar de constructores extensos o creación manual de objetos repetitivos, ya que estos aumentan la complejidad cuando existen configuraciones similares. Este patrón permite reutilizar estructuras base para reportes sin redefinir toda su configuración.

Composite

RF-028, RF-031 (estructura de zonas y asientos)

Se utilizó Composite para modelar la relación entre zonas y asientos.

Se eligió en lugar de estructuras planas basadas en listas o relaciones simples, ya que estas no representan adecuadamente jerarquías parte-todo. Tampoco se utilizó una agregación simple porque no permite tratar componentes individuales y compuestos de forma uniforme.

Decorator

RF-009, RF-035 (servicios adicionales en compras)

Se utilizó Decorator para la adición dinámica de servicios adicionales en la compra.

Se eligió en lugar de la herencia, ya que esta generaría una explosión de subclases para cada combinación de servicios (VIP, seguro, merchandising, parqueadero). También se evitó el uso de condicionales dentro de la clase Compra, ya que esto reduce la flexibilidad del sistema.

Adapter

RF-046 (generación de reportes PDF y CSV)

Se utilizó Adapter para integrar una librería externa de generación de reportes.

Se eligió en lugar de modificar la librería externa o acoplarla directamente al sistema, ya que esto no es posible o no es recomendable. El Adapter permite adaptar su interfaz al sistema interno sin afectar su arquitectura.

Strategy

RF-007, RF-034 (procesamiento de pagos)

Se utilizó Strategy para manejar diferentes métodos de pago.

Se eligió en lugar de estructuras condicionales como `switch` o `if-else`, ya que estas aumentan la complejidad cuando crecen los métodos de pago. Tampoco se utilizó herencia, ya que no permite cambiar el comportamiento dinámicamente en tiempo de ejecución.

Proxy

RF-034, RF-045 (control de acceso a compras)

Se utilizó Proxy para controlar el acceso a las operaciones de compra.

Se eligió en lugar de colocar validaciones directamente en la clase Compra o en múltiples servicios, ya que esto violaría el principio de responsabilidad única y generaría código disperso. El Proxy centraliza la validación de usuario antes de permitir operaciones sensibles.

Observer

RF-017, RF-041 (incidencias y notificaciones del sistema)

Se utilizó Observer para notificar cambios relevantes en el sistema.

Se eligió en lugar de llamadas directas entre clases, ya que esto genera alto acoplamiento. Tampoco se utilizó verificación constante del estado (polling), ya que es ineficiente y poco escalable.

State

RF-008, RF-032 (estados de compra y asientos)

Se utilizó State para manejar los distintos estados del sistema.

Se eligió en lugar de estructuras condicionales (`if-else` o `switch`) basadas en estado, ya que estas vuelven el código difícil de mantener. Tampoco se utilizó lógica de estado dentro de enums, ya que esto mezcla responsabilidades y reduce la extensibilidad.

CONCLUSIÓN

La implementación de estos patrones de diseño permite dar solución directa a los requisitos funcionales del sistema, garantizando una arquitectura modular, escalable y mantenible. Cada patrón fue seleccionado no solo por su utilidad general, sino por su capacidad de resolver problemas concretos definidos en los RF del proyecto, evitando alternativas más rígidas como condicionales extensos, herencia excesiva o alto acoplamiento entre clases.